

Experiment No.	3
Experiment Title	Regression Analysis using Linear and Regularized Models
GitHub Repository	https://github.com/RadeshL/Machine-Learning-Laboratory/blob/main/Mlassign3.ipynb
Student Name	L Radesh
Register Number	3122247001047
Faculty	Dr. Poreddy Ajay Kumar Reddy
Academic Year	2026–2027
Semester	VI
Submission Date	16 August, 2026

1 Objective

The objective of this experiment is to implement and compare linear and regularized regression models for predicting a continuous target variable. The experiment uses a real-world loan application dataset in which the target variable is the loan amount sanctioned.

The specific objectives are:

1. To perform exploratory data analysis on the loan application dataset.
2. To identify and handle missing values and outliers.
3. To preprocess numerical and categorical features using appropriate transformation, scaling, and encoding techniques.
4. To implement Linear Regression as a baseline model.
5. To implement Ridge, Lasso, and Elastic Net regression models.
6. To tune regularization hyperparameters using GridSearchCV and Randomized-SearchCV with 5-fold cross-validation.
7. To evaluate the regression models using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 score.
8. To visualize predicted values, residuals, training and validation errors, and regression coefficients.
9. To analyze the effect of regularization on model complexity, overfitting, underfitting, and the bias–variance trade-off.

2 Problem Statement

The problem is to predict the amount of loan that will be sanctioned to a customer based on information associated with a loan application.

The dataset contains both numerical and categorical information related to customers, their financial characteristics, employment information, property information, credit history, loan requirements, and other application attributes.

Let the input feature vector be represented by

$$\mathbf{x} = [x_1, x_2, \dots, x_p]^T$$

and let the target variable be

$$y = \text{Loan Sanction Amount (USD)}.$$

The objective is to learn a regression function

$$\hat{y} = f(\mathbf{x})$$

such that the predicted loan sanction amount is as close as possible to the actual sanctioned amount.

Four regression models are considered:

1. Linear Regression
2. Ridge Regression
3. Lasso Regression
4. Elastic Net Regression

The performance of the models is compared using regression error metrics and the effect of regularization is analyzed.

3 Dataset Description

The dataset used in this experiment is the *Predict Loan Amount Data* dataset referenced by the laboratory manual. It is a real-world regression dataset containing numerical and categorical features related to loan applications.

The target variable is **Loan Sanction Amount (USD)**.

3.1 Dataset Summary

Property	Description
Dataset Name	Predict Loan Amount Data
Dataset Source	Kaggle
Initial Number of Samples	30,000
Initial Number of Features/Columns	24
Target Variable	Loan Sanction Amount (USD)
Final Number of Columns	49
Number of Duplicate Rows	0
Train-Test Split	80% training, 20% testing

Random State	42
Cross-Validation	5-Fold Cross-Validation

3.2 Original Dataset Features

The original dataset contains the following attributes:

- Customer ID
- Name
- Gender
- Age
- Income (USD)
- Income Stability
- Profession
- Type of Employment
- Location
- Loan Amount Request (USD)
- Current Loan Expenses (USD)
- Expense Type 1
- Expense Type 2
- Dependents
- Credit Score
- No. of Defaults
- Has Active Credit Card
- Property ID
- Property Age
- Property Type
- Property Location
- Co-Applicant
- Property Price
- Loan Sanction Amount (USD)

3.3 Target Variable

The prediction target is:

Loan Sanction Amount (USD)

This is a continuous numerical variable, making the problem a supervised regression problem.

4 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was performed before model training to understand the structure and quality of the dataset.

The initial dataset contains 30,000 rows and 24 columns. No duplicate rows were detected.

4.1 Dataset Overview

The first five rows of the original dataset showed information about customers, income, employment, loan requirements, credit score, property details, and the loan amount sanctioned.

The target variable is directly available in the dataset as **Loan Sanction Amount (USD)**.

4.2 Missing Value Analysis

The initial missing-value analysis identified missing observations in several columns.

The main columns containing missing values were:

- Gender
- Income (USD)
- Income Stability
- Type of Employment
- Current Loan Expenses (USD)
- Dependents
- Credit Score
- Has Active Credit Card
- Property Age
- Property Location
- Loan Sanction Amount (USD)

No column contained more than 50% missing values. Therefore, no columns were removed due to excessive missing data.

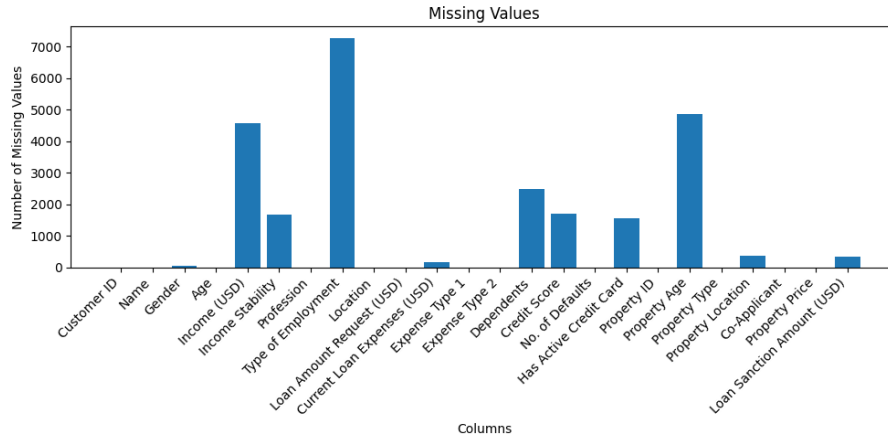


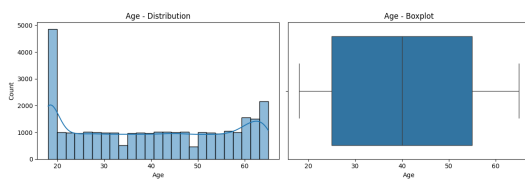
Figure 1: Missing-value distribution across the original dataset columns.

Inference:

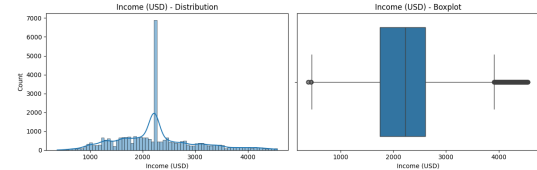
The missing-value analysis shows that missing observations are distributed across several numerical and categorical attributes. No feature exceeded the 50% missing-value threshold, allowing all original features to be retained at this stage. Numerical missing values were subsequently replaced using their median values, while categorical missing values were replaced using their mode. This prevents loss of a substantial number of samples while maintaining a complete dataset for subsequent processing.

4.3 Statistical and Feature Distribution Analysis

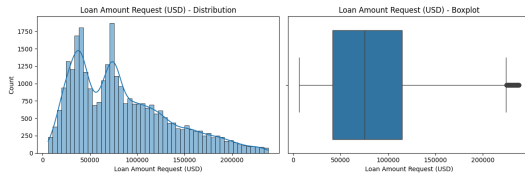
These visualizations were used to inspect the distributions, central tendencies, spread, skewness, and potential outliers in the numerical features.



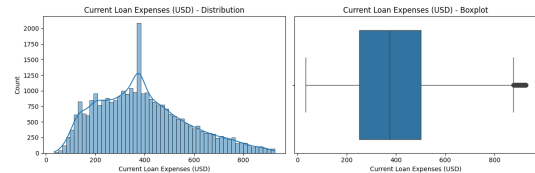
(a) Age



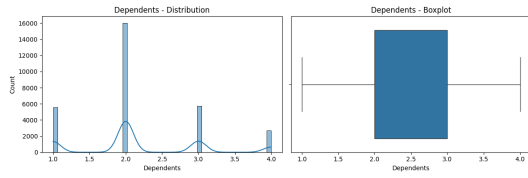
(b) Income (USD)



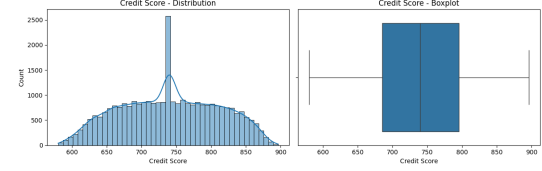
(c) Loan Amount Request (USD)



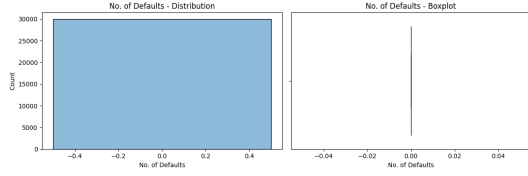
(d) Current Loan Expenses (USD)



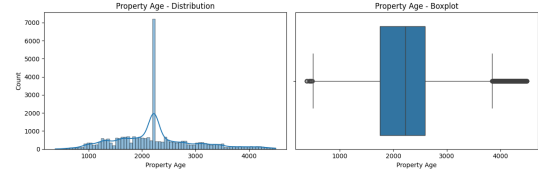
(a) Dependents



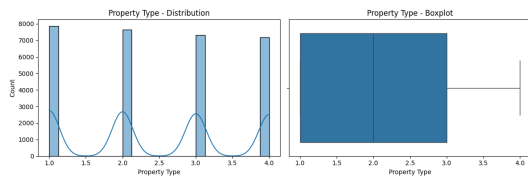
(b) Credit Score



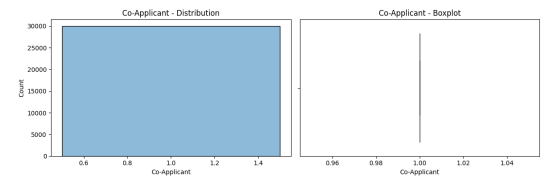
(c) No. of Defaults



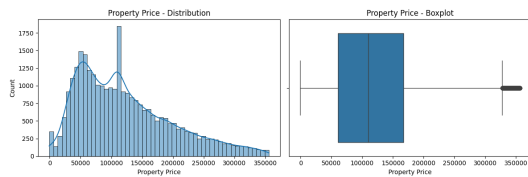
(d) Property Age



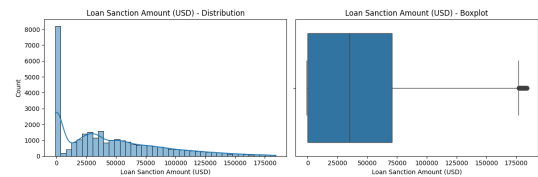
(a) Property Type



(b) Co-Applicant



(c) Property Price



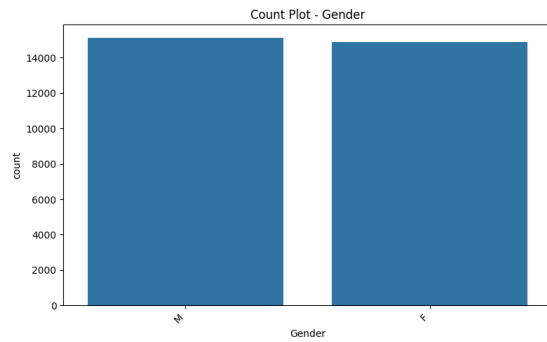
(d) Loan Sanction Amount (USD)

Inference:

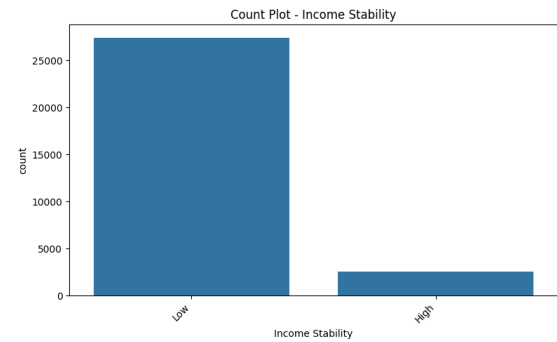
The histograms provide information about the distribution, central tendency, and skewness of the numerical variables, while the corresponding boxplots help identify potential outliers. The plots show that the numerical features have different scales and distribution characteristics, which makes feature standardization important before applying the regression algorithms. The boxplots also reveal observations lying away from the central range of some variables, motivating the use of the IQR method for outlier detection and treatment. These preprocessing steps help produce a more consistent feature representation for the subsequent regression models.

4.4 Categorical Feature Analysis

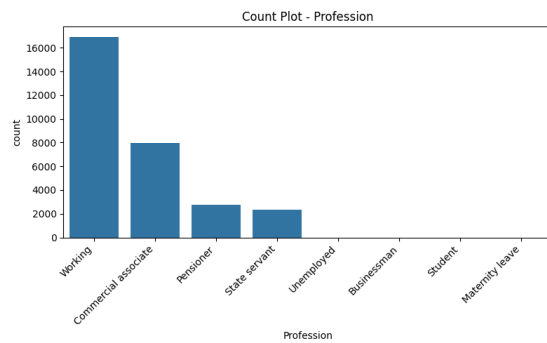
Count plots were generated for categorical features with a manageable number of categories. High-cardinality attributes such as Customer ID and Name were excluded from categorical visualization and encoding because they contain too many unique categories.



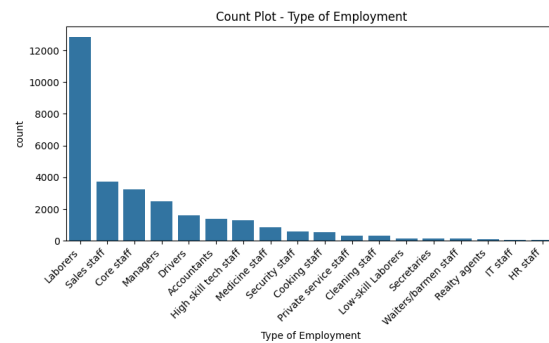
(a) Gender



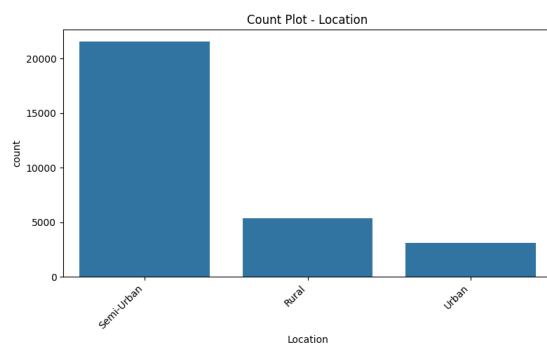
(b) Income Stability



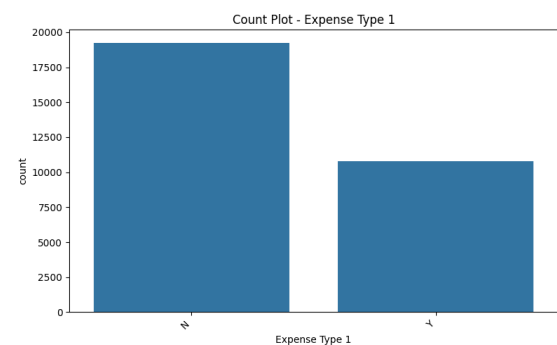
(c) Profession



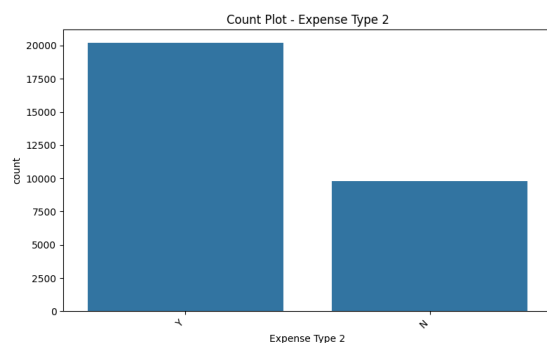
(d) Type of Employment



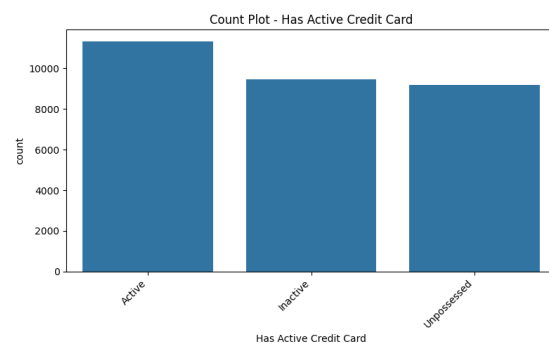
(a) Location



(b) Expense Type 1



(c) Expense Type 2



(d) Has Active Credit Card



(a) Property Location

Inference:

The categorical plots show the distribution of observations across the different categories. Some categorical variables contain only two categories and are therefore suitable for binary encoding, while variables with multiple categories require one-hot encoding. Customer ID and Name were treated as high-cardinality attributes and were not encoded.

5 Data Preprocessing

The preprocessing procedure consists of missing-value handling, outlier treatment, numerical transformation, categorical encoding, and removal of non-predictive identifier columns before model training.

5.1 Missing Value Handling

For numerical columns, missing values were replaced using the median:

$$x_{\text{missing}} = \text{median}(x).$$

For categorical columns, missing values were replaced using the most frequently occurring category:

$$x_{\text{missing}} = \text{mode}(x).$$

This procedure resulted in zero remaining missing values after preprocessing.

5.2 Outlier Detection

Outliers were detected using the Interquartile Range (IQR) method.

The first quartile and third quartile are represented by Q_1 and Q_3 . The IQR is

$$IQR = Q_3 - Q_1.$$

The lower and upper limits were calculated as

$$L = Q_1 - 1.5(IQR)$$

and

$$U = Q_3 + 1.5(IQR).$$

Observations outside this interval were treated as outliers.

The implementation used a threshold of 15% to distinguish between a relatively small number of outliers and a more substantial proportion.

For variables with a small outlier percentage, outliers were replaced by the median. For variables with a higher proportion of outliers, the values were clipped to the IQR boundaries.

5.3 Observed Outlier Treatment

Feature	Outliers	Percentage	Treatment
Age	0	0%	No treatment
Income (USD)	1874	6.25%	Median replacement
Loan Amount Request (USD)	752	2.51%	Median replacement
Current Loan Expenses (USD)	874	2.91%	Median replacement
Dependents	432	1.44%	Median replacement
Credit Score	0	0%	No treatment
No. of Defaults	5818	19.39%	IQR boundary clipping
Property ID	0	0%	No treatment
Property Age	1930	6.43%	Median replacement
Property Type	0	0%	No treatment
Co-Applicant	4484	14.95%	Median replacement
Property Price	863	2.88%	Median replacement
Loan Sanction Amount (USD)	453	1.51%	Median replacement

5.4 Numerical Feature Scaling

The skewness for each numerical feature was calculated.

The transformation rule was:

- If $|\text{skewness}| > 1$ and the feature was non-negative, $\log_{1p}$ transformation was considered.
- If $|\text{skewness}| > 1$ and negative values existed, StandardScaler was used.
- For features with $|\text{skewness}| \leq 1$, StandardScaler was applied.

In the actual dataset, the reported skewness values were within the reasonable range for all listed numerical features. Therefore, StandardScaler was applied to the numerical variables.

Standardization is given by

$$z = \frac{x - \mu}{\sigma},$$

where μ is the feature mean and σ is its standard deviation.

5.5 Categorical Encoding

Binary categorical features were encoded using binary mappings.

For example, the notebook encoded:

- Gender
- Income Stability
- Expense Type 1
- Expense Type 2

Categorical features with more than two categories were transformed using one-hot encoding with the first category dropped.

The following variables were one-hot encoded:

- Profession
- Type of Employment
- Location
- Has Active Credit Card
- Property Location

Customer ID and Name were not encoded because they have high cardinality.

5.6 Final Dataset

After preprocessing, the dataset contained:

30,000 samples $\times$ 49 columns.

The identifier columns Customer ID and Name were subsequently excluded from the model feature matrix.

The final model therefore operates on numerical and encoded categorical predictors.

6 Train-Test Split

The dataset was divided into training and testing subsets using an 80:20 ratio. Thus, approximately 24,000 observations were used for training and 6,000 observations were used for testing.

7 Machine Learning Algorithms

Four regression algorithms were implemented:

1. Linear Regression
2. Ridge Regression
3. Lasso Regression
4. Elastic Net Regression

7.1 Linear Regression

Linear Regression models the target variable as a linear combination of the input features. The mathematical formulation is

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_p x_p.$$

The model estimates the coefficients by minimizing the residual sum of squares:

$$\min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

Linear Regression provides a simple and interpretable baseline.

7.1.1 Advantages

- Simple to implement.
- Computationally efficient.
- Easy to interpret.
- Provides a useful baseline for comparison.

7.1.2 Limitations

- Sensitive to multicollinearity.
- Can overfit when many correlated features are present.
- Assumes a linear relationship between predictors and target.

7.2 Ridge Regression

Ridge Regression introduces an L_2 regularization penalty. Its objective function is

$$\min_{\beta} \left[\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^p \beta_j^2 \right].$$

The regularization term penalizes large coefficient magnitudes. A larger value of α produces stronger regularization.

7.2.1 Advantages

- Reduces coefficient magnitudes.
- Helps reduce variance.
- Can improve generalization in the presence of correlated features.

7.2.2 Limitations

- Does not generally force coefficients exactly to zero.
- Therefore, it does not perform explicit feature selection.

7.3 Lasso Regression

Lasso Regression uses an L_1 regularization penalty.

Its objective function is

$$\min_{\beta} \left[\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \alpha \sum_{j=1}^p |\beta_j| \right].$$

The L_1 penalty can drive some coefficients exactly to zero.

7.3.1 Advantages

- Performs automatic feature selection.
- Produces sparse models.
- Can improve interpretability.

7.3.2 Limitations

- Can discard useful correlated features.
- Performance is sensitive to the regularization parameter.

7.4 Elastic Net Regression

Elastic Net combines L_1 and L_2 regularization.

Its objective function can be represented as

$$\min_{\beta} \left[\sum_{i=1}^n (y_i - \hat{y}_i)^2 + \alpha \left(\rho \sum_{j=1}^p |\beta_j| + (1 - \rho) \sum_{j=1}^p \beta_j^2 \right) \right],$$

where ρ is the L1 ratio.

When ρ is close to 1, the model behaves more like Lasso, while lower values increase the contribution of Ridge-style regularization.

7.4.1 Advantages

- Combines feature selection and coefficient shrinkage.
- Can handle correlated predictors more effectively than pure Lasso.
- Provides a flexible balance between L1 and L2 penalties.

7.4.2 Limitations

- Has more hyperparameters to tune.
- Can require greater computational time than basic Linear Regression.

8 Experimental Setup

Parameter	Configuration
Python Version	3.13.3
Notebook Kernel	Python 3
Machine Learning Library	Scikit-learn
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Seaborn
IDE	Jupyter Notebook
Operating System	Windows
Hardware	None
Random State	42
Train-Test Ratio	80:20
Cross-Validation	5-Fold

9 Hyperparameter Selection

The regularized regression models were tuned using both GridSearchCV and RandomizedSearchCV.

The search was performed using 5-fold cross-validation and `neg_mean_squared_error` as the optimization score.

9.1 Search Spaces

Model	Search Space
Ridge	$\alpha \in \{0.01, 0.1, 1, 10, 100\}$
Lasso	$\alpha \in \{0.001, 0.01, 0.1, 1, 10\}$
Elastic Net	$\alpha \in \{0.01, 0.1, 1, 10\}$ and $l1_ratio \in \{0.2, 0.5, 0.8\}$

9.2 Grid Search Results

Model	Best Parameters	Test RMSE	Test R^2
Ridge	$\alpha = 100$	0.712176	0.490816
Lasso	$\alpha = 0.001$	0.712226	0.490744

Elastic Net	$\alpha = 0.01$, $l1_ratio = 0.2$	0.711781	0.491379
-------------	-------------------------------------	----------	----------

The best-performing tuned configuration according to the test-set R^2 was Elastic Net with

$$\alpha = 0.01, \quad l1_ratio = 0.2.$$

10 Results

10.1 Baseline and Regularized Model Performance

Model	MAE	MSE	RMSE	R^2	Training Time (s)
Linear Regression	0.500173	0.507597	0.712458	0.490412	0.072413
Ridge Regression	0.500169	0.507593	0.712455	0.490416	0.016880
Lasso Regression	0.523793	0.526109	0.725333	0.471827	0.074277
Elastic Net	0.513970	0.513546	0.716621	0.484439	0.091216

10.2 Tuned Model Performance

Model	MAE	MSE	RMSE	R^2	Training Time (s)
Grid Ridge	0.500239	0.507194	0.712176	0.490816	0.538046
Grid Lasso	0.499948	0.507265	0.712226	0.490744	4.881348
Grid Elastic Net	0.500990	0.506633	0.711781	0.491379	5.851213

10.3 Randomized Search Performance

Model	Best Parameters	MAE	RMSE	R^2	Time (s)
Randomized Ridge	$\alpha = 100$	0.500239	0.712176	0.490816	0.512971
Randomized Lasso	$\alpha = 0.001$	0.499948	0.712226	0.490744	4.906287
Randomized Elastic Net	$\alpha = 0.01$, $l1_ratio = 0.2$	0.500990	0.711781	0.491379	5.545601

An important observation is that GridSearchCV and RandomizedSearchCV selected the same best parameters for all three regularized models. Consequently, both approaches produced identical test-set performance in this experiment.

11 Performance Metrics

11.1 Mean Absolute Error

Mean Absolute Error is defined as

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

It measures the average absolute difference between the actual and predicted values. Lower MAE indicates better prediction accuracy.

11.2 Mean Squared Error

Mean Squared Error is

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

MSE penalizes larger errors more strongly than MAE.

11.3 Root Mean Squared Error

RMSE is

$$RMSE = \sqrt{MSE}.$$

It represents the square root of the average squared prediction error.

11.4 R^2 Score

The coefficient of determination is

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}.$$

An R^2 value closer to 1 indicates that the model explains a greater proportion of the variance in the target.

The best tuned model obtained an R^2 of approximately

$$R^2 = 0.4914.$$

This indicates that approximately 49.14% of the variance in the preprocessed target is explained by the tuned Elastic Net model on the test set.

11.5 Important Scale Consideration

Applies `StandardScaler` to all numerical columns selected during preprocessing, including the target variable `Loan Sanction Amount (USD)`.

Therefore, the reported MAE, MSE, and RMSE values are measured on the standardized target scale rather than directly in USD.

Consequently, the numerical error values should not be interpreted as dollar amounts.

For a production-quality regression experiment, the preferred approach would be to scale only the predictor variables and retain the original target scale, or alternatively inverse-transform predictions before reporting errors in USD.

12 Result Visualization

1. Predicted versus actual values
2. Residual plots
3. Training versus validation/test RMSE
4. Coefficient comparison

12.1 Predicted versus Actual Values

A predicted-versus-actual plot compares the model predictions with the true target values. The ideal relationship is represented by

$$\hat{y} = y.$$

Thus, points close to the diagonal line indicate accurate predictions.

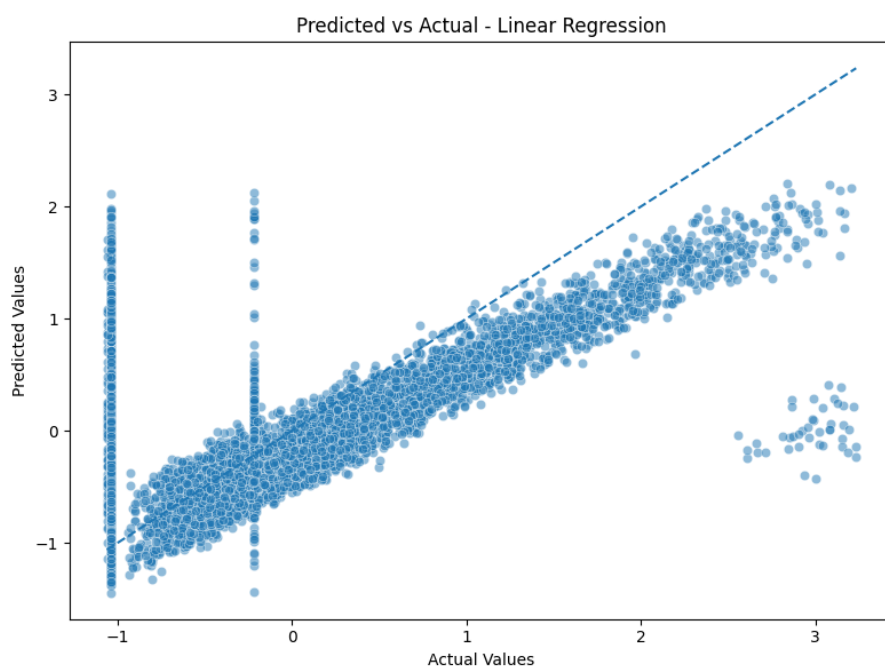


Figure 8: Predicted versus actual values for Linear Regression.

Inference:

The predicted-versus-actual visualization shows the degree to which the predicted loan sanction amounts follow the actual values. Points close to the diagonal reference line indicate accurate predictions. The spread of points around the line demonstrates that the linear model does not explain all variation in the target. This is consistent with its test R^2 of approximately 0.49.

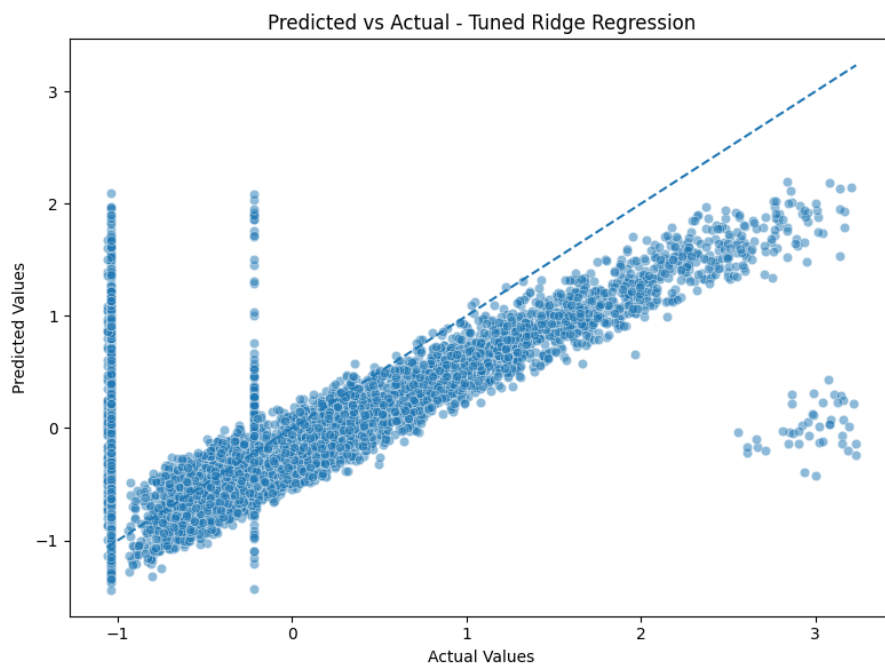


Figure 9: Predicted versus actual values for tuned Ridge Regression.

Inference:

The tuned Ridge predictions follow a pattern similar to the baseline Linear Regression model. Regularization produces only a small numerical improvement in the test metrics. The resulting R^2 is approximately 0.491, indicating a modest improvement in generalization.

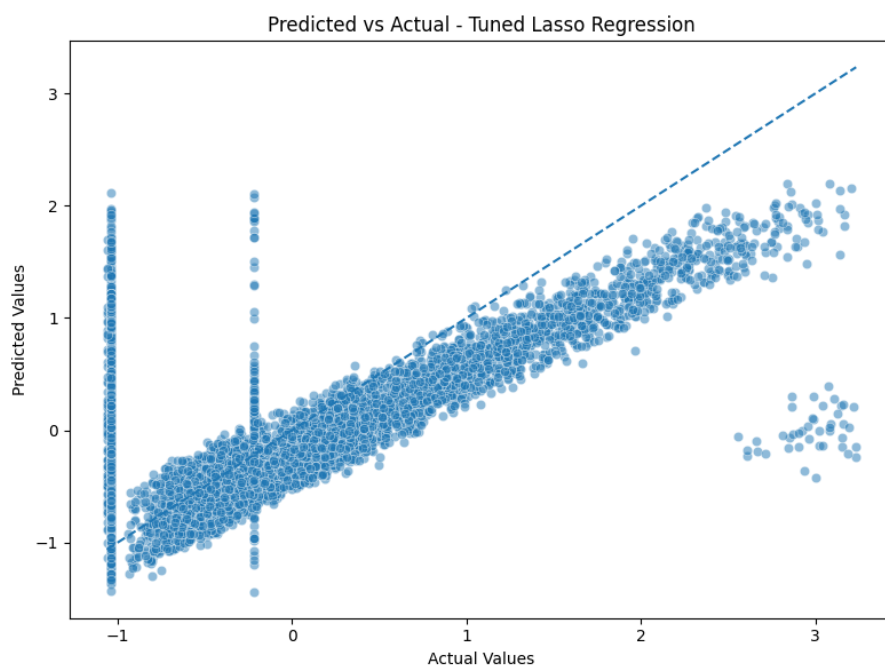


Figure 10: Predicted versus actual values for tuned Lasso Regression.

Inference:

The tuned Lasso model produces predictions that are broadly similar to the other linear models. Its test MAE is slightly lower than that of tuned Ridge and Elastic Net, although its RMSE and R^2 are marginally worse than those of Elastic Net. The relatively small difference between the models indicates that regularization does not dramatically change the prediction structure for this dataset.

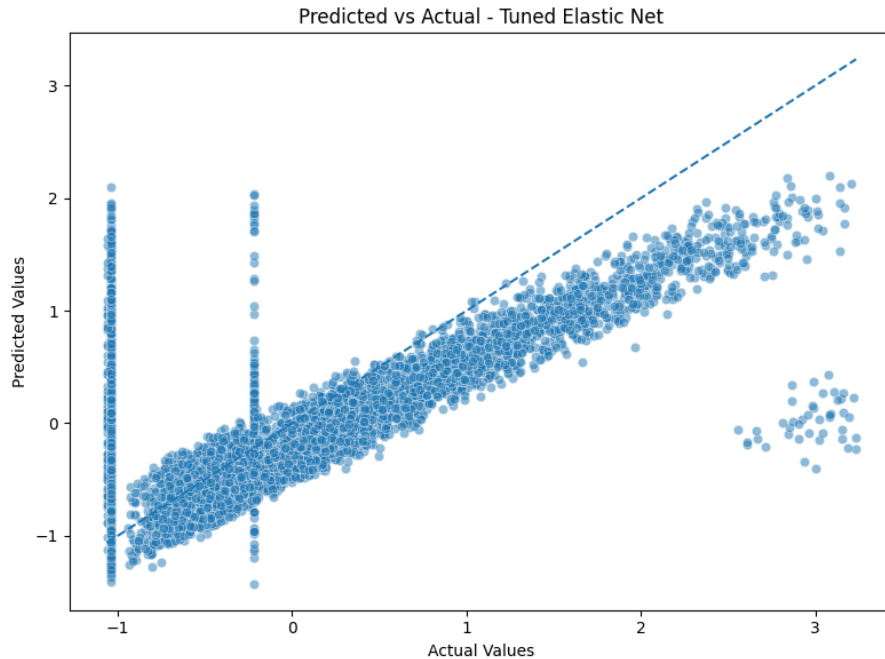


Figure 11: Predicted versus actual values for tuned Elastic Net Regression.

Inference:

Elastic Net provides the strongest overall test-set performance among the tuned models. It obtains the lowest RMSE and highest R^2 in the reported results. The prediction scatter therefore provides visual support for the numerical conclusion that Elastic Net is the preferred tuned model in this experiment.

12.2 Residual Analysis

A residual is defined as

$$e_i = y_i - \hat{y}_i.$$

An ideal residual plot should show residuals distributed approximately randomly around zero without a strong systematic pattern.



Figure 12: Residual plot for Linear Regression.

Inference:

The residual plot is used to determine whether systematic prediction errors remain after fitting the regression model. Residuals distributed around zero indicate that the model captures the main linear relationship. Any visible structure or changing spread would indicate that some information in the data remains unexplained by the model.

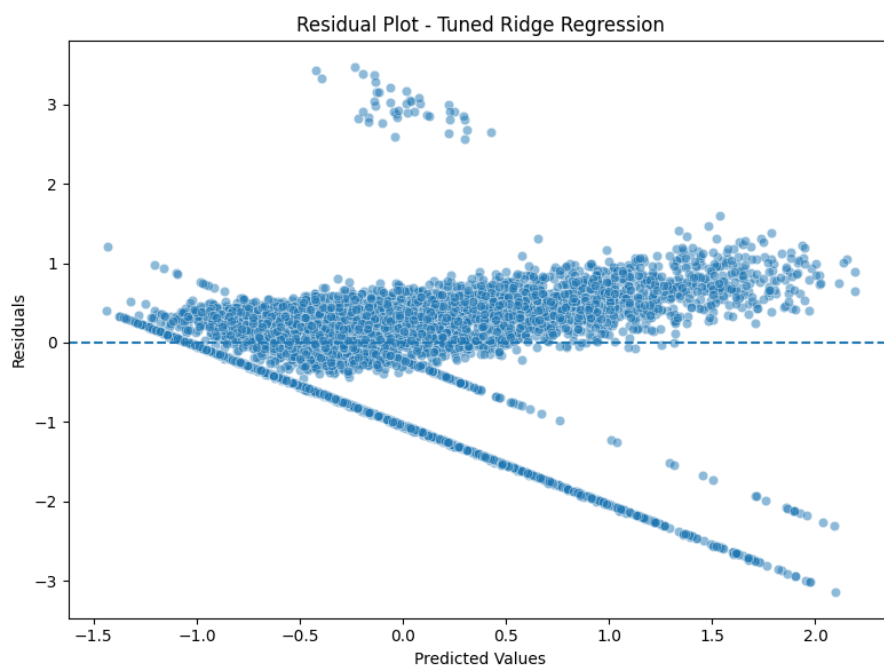


Figure 13: Residual plot for tuned Ridge Regression.

Inference:

The tuned Ridge residual plot provides a diagnostic view of the effect of L_2 regularization. A similar residual pattern to Linear Regression indicates that Ridge has not substantially changed the underlying prediction relationship. Its primary contribution is coefficient shrinkage and modest variance control.

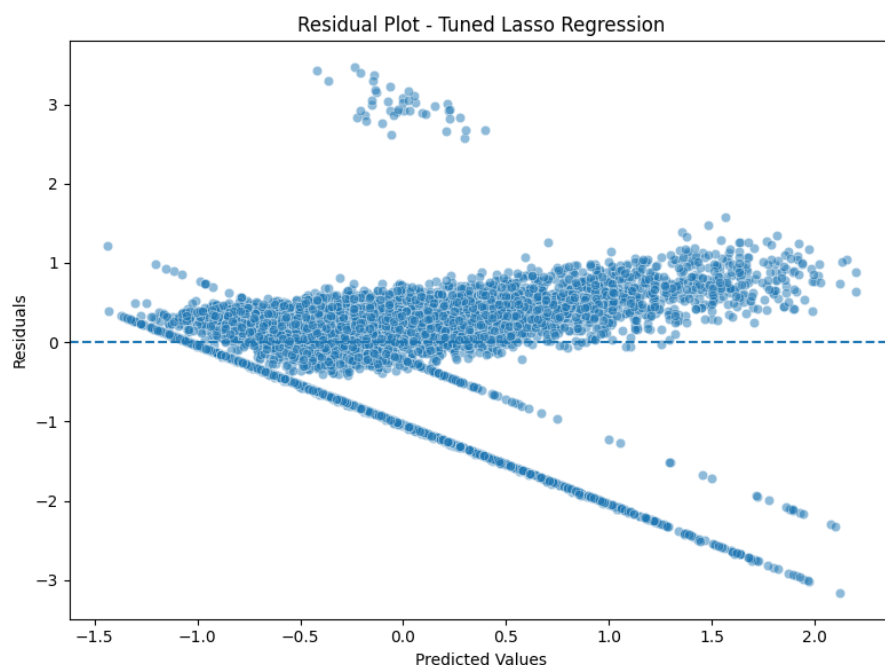


Figure 14: Residual plot for tuned Lasso Regression.

Inference:

The Lasso residual plot shows the remaining prediction errors after introducing L_1 regularization. Lasso encourages sparse coefficients, which can simplify the model. However, its overall test R^2 remains slightly below that of the tuned Elastic Net model.

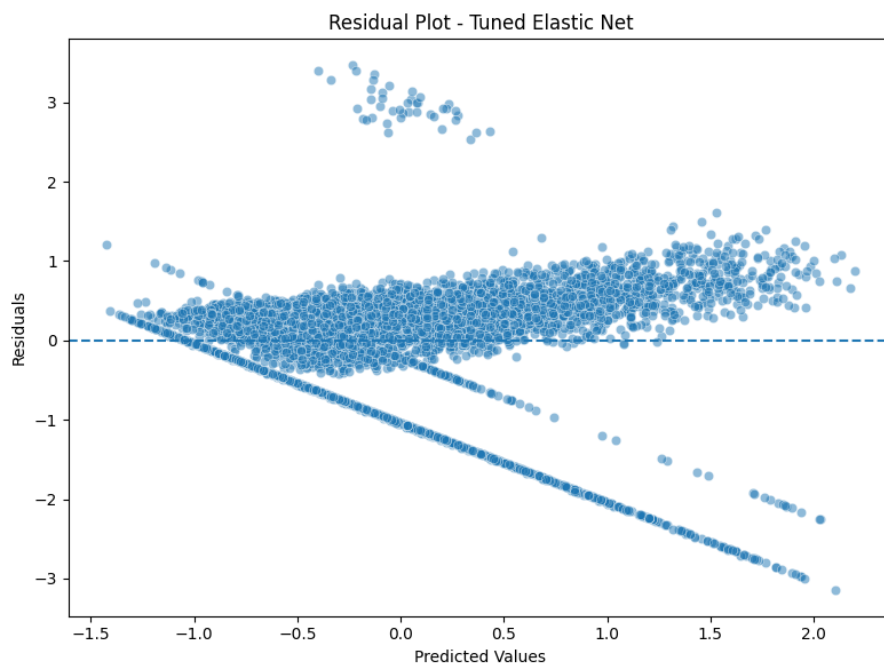


Figure 15: Residual plot for tuned Elastic Net Regression.

Inference:

The Elastic Net residual plot is used to assess whether the combined regularization approach leaves systematic patterns in the errors. The model's relatively low RMSE and highest test R^2 among the tuned models suggest that its balance between coefficient shrinkage and sparsity is effective for this dataset.

12.3 Training Error versus Validation/Test Error

Compared training RMSE and test-set RMSE for the four models.



Figure 16: Comparison of training RMSE and validation/test RMSE across models.

Inference:

The training-error versus validation/test-error plot is useful for detecting possible overfitting and underfitting. A very large difference between training and validation error would indicate that the model is fitting the training data substantially better than unseen data. The regularized models provide a mechanism for controlling this gap by penalizing excessive coefficient magnitudes. In this implementation, the plot compares the training error with the held-out test error rather than a separately stored cross-validation validation-error curve.

12.4 Coefficient Comparison

The regression coefficient for a feature represents its contribution to the predicted target while holding the other features constant.

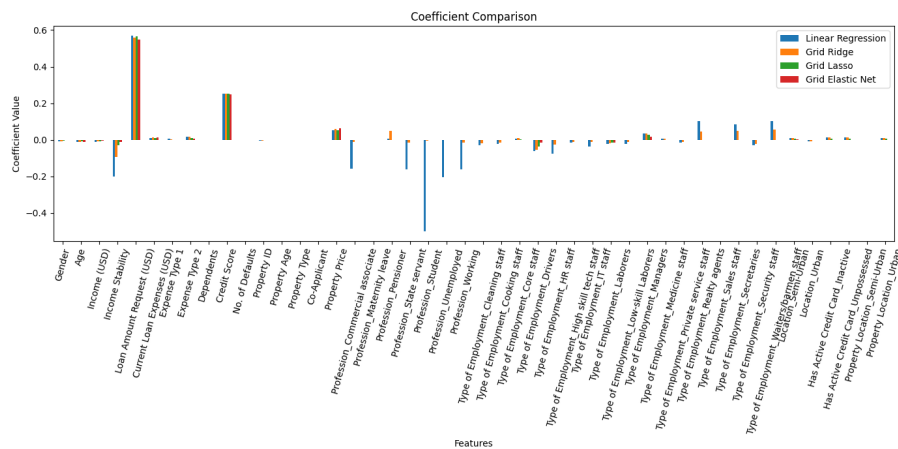


Figure 17: Coefficient comparison among Linear, Ridge, Lasso, and Elastic Net models.

Inference:

The coefficient comparison illustrates the effect of regularization on model parameters. Ridge reduces coefficient magnitudes through its L_2 penalty, while Lasso can drive coefficients exactly to zero because of its L_1 penalty. Elastic Net combines both effects. Therefore, the coefficient plot provides a direct visualization of the trade-off between model complexity and feature contribution.

13 Overfitting and Underfitting Analysis

Overfitting occurs when a model learns the training data too closely and performs significantly worse on unseen data. Underfitting occurs when the model is too simple to capture the underlying relationships.

13.1 Linear Regression

Linear Regression provides the baseline performance.

Its test-set performance was:

$$MAE = 0.500173$$

$$RMSE = 0.712458$$

$$R^2 = 0.490412.$$

The baseline model provides a useful reference for evaluating whether regularization produces improvements.

13.2 Effect of Ridge Regularization

The baseline Ridge model has nearly identical performance to Linear Regression.

The tuned Ridge model improves the test R^2 from approximately

$$0.490412$$

to

$$0.490816.$$

The improvement is small, indicating that Ridge regularization does not dramatically change the predictive capability of the model.

13.3 Effect of Lasso Regularization

The initial Lasso model performs worse than Linear Regression:

$$R^2 = 0.471827.$$

After hyperparameter tuning, the Lasso model improves substantially:

$$R^2 = 0.490744.$$

This demonstrates the importance of selecting an appropriate regularization strength.

13.4 Effect of Elastic Net Regularization

The initial Elastic Net configuration achieves

$$R^2 = 0.484439.$$

After tuning, its performance improves to

$$R^2 = 0.491379.$$

Thus, hyperparameter optimization produces the strongest test-set performance among the evaluated models.

14 Bias–Variance Analysis

The bias–variance trade-off describes the relationship between model complexity, systematic error, and sensitivity to the training data.

14.1 Linear Regression

Linear Regression has relatively low model complexity and therefore provides a useful baseline with relatively controlled variance. However, its linear structure can introduce bias when the underlying relationship between the features and target is more complex.

14.2 Ridge Regression

Ridge adds an L_2 penalty that shrinks coefficients toward zero.

Increasing α increases the strength of the penalty and generally reduces model variance at the cost of potentially increasing bias.

The tuned Ridge model selected

$$\alpha = 100.$$

This relatively large regularization value indicates that strong coefficient shrinkage produced the best result within the tested search space.

14.3 Lasso Regression

Lasso applies an L_1 penalty and can produce sparse coefficient vectors.

The tuned Lasso model selected

$$\alpha = 0.001.$$

The small value suggests that strong L1 regularization was not preferred within the tested search space.

The model can nevertheless perform feature selection by shrinking some coefficients to zero.

14.4 Elastic Net

Elastic Net provides a compromise between L1 and L2 regularization.

The selected configuration was

$$\alpha = 0.01$$

and

$$l1_ratio = 0.2.$$

A value of 0.2 indicates a stronger contribution from the L2 component than the L1 component. This configuration achieved the highest test R^2 among the tuned models.

15 Grid Search and Randomized Search Comparison

Both GridSearchCV and RandomizedSearchCV were implemented for Ridge, Lasso, and Elastic Net.

The important observation is that both methods selected identical best parameters:

Model	Grid Search	Randomized Search
Ridge	$\alpha = 100$	$\alpha = 100$
Lasso	$\alpha = 0.001$	$\alpha = 0.001$
Elastic Net	$\alpha = 0.01, l1_ratio = 0.2$	$\alpha = 0.01, l1_ratio = 0.2$

Since the randomized searches sampled the complete available search spaces in this particular experiment, they produced the same best configurations as GridSearchCV.

The measured execution times were approximately:

- Ridge Grid Search: 0.538 seconds
- Ridge Randomized Search: 0.513 seconds
- Lasso Grid Search: 4.881 seconds
- Lasso Randomized Search: 4.906 seconds
- Elastic Net Grid Search: 5.851 seconds
- Elastic Net Randomized Search: 5.546 seconds

The small timing differences demonstrate that randomized search can have a computational advantage when fewer configurations are sampled, although the difference depends on the search space and number of iterations.

16 Discussion

The experiment demonstrates the implementation and comparison of four linear regression approaches for predicting loan sanction amounts.

The baseline Linear Regression model achieved an R^2 score of approximately 0.4904. Ridge produced an almost identical baseline result, while the initial Lasso and Elastic Net configurations performed slightly worse.

Hyperparameter tuning improved the performance of all regularized models. The best Ridge parameter was $\alpha = 100$, while Lasso selected $\alpha = 0.001$. Elastic Net selected $\alpha = 0.01$ and $l1_ratio = 0.2$.

Among the tuned models, Elastic Net achieved the best test performance:

$$MAE = 0.500990,$$

$$MSE = 0.506633,$$

$$RMSE = 0.711781,$$

and

$$R^2 = 0.491379.$$

The improvement over baseline Linear Regression is relatively small. Therefore, regularization improves the result, but it does not completely solve the limitations of a linear model for this dataset.

16.1 Strengths

The experiment has several strengths:

- A complete preprocessing procedure was implemented.
- Both numerical and categorical variables were handled.
- Missing values were systematically imputed.
- Outliers were explicitly detected and treated.
- Four regression algorithms were compared.
- Both GridSearchCV and RandomizedSearchCV were implemented.
- Five-fold cross-validation was used for hyperparameter selection.
- Multiple regression metrics were reported.
- Model predictions, residuals, errors, and coefficients were visualized.

16.2 Limitations and Improvements

All four models considered are linear regression models. If the underlying relationship between loan characteristics and sanctioned amount is nonlinear, models such as Random Forest Regression, Gradient Boosting, XGBoost, or other nonlinear approaches could potentially provide better performance.

17 Observations

The major observations from the experiment are:

1. The dataset contains 30,000 loan application records and multiple numerical and categorical attributes.
2. Missing values were present in several columns, but no column exceeded the 50% missing-value threshold.
3. Numerical missing values were handled using median imputation, while categorical missing values were handled using mode imputation.
4. Outliers were identified using the IQR method.
5. Numerical features were standardized using StandardScaler.
6. Binary categorical variables were binary encoded, while multi-category variables were one-hot encoded.
7. The final preprocessed dataset contained 49 columns.
8. Linear Regression achieved an R^2 score of approximately 0.4904.

9. The initial Lasso and Elastic Net models performed worse than the baseline Linear Regression model.
10. Hyperparameter tuning improved the performance of the regularized models.
11. GridSearchCV and RandomizedSearchCV selected identical best hyperparameters for all three regularized models.
12. Elastic Net achieved the highest test R^2 of approximately 0.4914.
13. Elastic Net also achieved the lowest test RMSE among the tuned models.
14. The improvement from regularization was relatively small, suggesting that the predictive capability of the tested linear models is limited by the feature-target relationship.

18 Conclusion

This experiment implemented Linear Regression, Ridge Regression, Lasso Regression, and Elastic Net Regression for predicting loan sanction amounts using a real-world loan application dataset.

The dataset was first explored and preprocessed through missing-value imputation, IQR-based outlier handling, numerical standardization, and categorical encoding. The resulting dataset contained 30,000 samples and 49 processed columns, from which the identifier columns Customer ID and Name were excluded before model training.

Linear Regression was used as the baseline model. Ridge, Lasso, and Elastic Net were subsequently trained to investigate the effect of regularization.

Hyperparameter tuning was performed using both GridSearchCV and RandomizedSearchCV with five-fold cross-validation. The best parameters were $\alpha = 100$ for Ridge, $\alpha = 0.001$ for Lasso, and $\alpha = 0.01$, $l1_ratio = 0.2$ for Elastic Net.

The tuned Elastic Net model achieved the best test-set performance with

$$MAE = 0.500990,$$

$$MSE = 0.506633,$$

$$RMSE = 0.711781,$$

and

$$R^2 = 0.491379.$$

Thus, Elastic Net is selected as the best model among the evaluated configurations.

However, the improvement over the baseline Linear Regression model is small. This indicates that regularization provides only a modest benefit for this particular feature representation. More advanced nonlinear regression methods, improved feature engineering, and a leakage-free preprocessing pipeline could be explored to improve predictive performance.

Finally, the experiment demonstrates the importance of model regularization, hyperparameter tuning, cross-validation, and error analysis when developing regression models for real-world prediction tasks.

19 References

- [1] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] Scikit-learn Documentation, “Linear Models.”
- [3] Scikit-learn Documentation, “Tuning the Hyper-Parameters of an Estimator.”
- [4] Kaggle, “Predict Loan Amount Data,” dataset used for regression analysis.